

1. **2-color vs 3-color.** Recordeu que un graf no dirigit $G = (V, E)$ és *bipartit* si existeix una partició de V en V_1, V_2 de manera que tota aresta $uv \in E$ satisfà o bé $u \in V_1 \wedge u \in V_2$ o bé $u \in V_2 \wedge u \in V_1$. Fixeu-vos que dir que un graf és bipartit és equivalent a dir que és 2-colorable (2-COLOR). Definim $\text{BIPARTITE GRAPH} = \{\langle G \rangle \mid G \text{ és bipartit}\}$.

Demostreu que $\text{BIPARTITE GRAPH} \in P$.

$\langle G \rangle$ graf com entrada per a una estructura de dades

Un DFS i compararem veïns un vèrtex són dif $O(|V| + |E|)$ (matriu)
 $O(|V|^2)$ (llista)

2. **DNF-Sat vs CNF-Sat.** Demostreu que els problemes següents pertanyen a la classe P:

- (a) Donada una fórmula booleana en *Forma Normal Conjuntiva (CNF)* on cada clàusula té com a màxim dos literals, decidir si és satisfactible.
- (b) Donada una fórmula booleana en *Forma Normal Disjuntiva (DNF)*, decidir si és satisfactible.
- (c) Una fórmula booleana en CNF es pot transformar en una fórmula booleana en DNF equivalent? Si es pot, en quant temps?

CNF $(x \vee y) \wedge (z \vee w)$ DNF $(x \wedge y) \vee (z \wedge w)$ 2-SAT !!

a) Sí, que no surti $(x, \neg x)$, lineal

b) Sí, mirem una clàusula true, lineal

c) Sí, temps exponencial, es pot transformar CNF de mida n i es surt una DNF de mida exponencial! (entrada $\geq$ sortida)

3. **Eulerian graph vs Hamiltonian graph** Recordeu que un recorregut eulerià en un graf no dirigit és un cycle que pot visitar repetides vegades un mateix vèrtex i ha d'utilitzar cada aresta exactament una vegada. Diem que un graf és *eulerià* si conté un recorregut eulerià.

Demostreu que

$\text{EULERIAN GRAPH} = \{ \langle G \rangle \mid G \text{ és eulerià} \} \in P.$

cycle !!! (no val impar)

Dissenyeu un algorisme eficient que en el cas que el graf sigui eulerià ens retorni un recorregut eulerià.

$\langle G \rangle$ flinto obj

És eulerià? Sí, mirant graus de cada vèrtex, i

o Sí $\Leftrightarrow$ 2 impar X

tots parells, $\left(\begin{array}{l} \text{començ} \\ 0 \\ \text{si no} \\ \text{ho és} \\ \text{vèrtex} \\ \text{aïllat} \end{array} \right)$

Comi

- Si 2 impar X

Comencem per vèrtex impar, creem vector per a tots els vèrtexs, si visitem una aresta la marquem, ens aneguem a ocular o l'impar!

- Si tots parells

Fem DFS i vector de booleans per a cada aresta, si visitem una aresta la indiquem com visitada.

(cycle $\Rightarrow$ acaba on comença)

Fem varios DFSs

4. **Shortest path vs longest path** Considerem les dues versions següents de *Shortest Path* i *Longest Path*, respectivament, per a grafs no dirigits:

$\text{SPATH} = \{ \langle G, a, b, k \rangle \mid G \text{ conté un camí simple de } a \text{ a } b \text{ de longitud menor o igual que } k \}$

$\text{LPATH} = \{ \langle G, a, b, k \rangle \mid G \text{ conté un camí simple de } a \text{ a } b \text{ de longitud més gran o igual que } k \}$.

(a) Demostreu que $\text{SPATH} \in P$.

(b) Demostreu que $\text{LPATH} \in \text{NP}$. Demostreu que si $\text{LPATH} \in P$ aleshores el problema del camí Hamiltonià en graf no dirigits també seria a P .

a) Amb *perce* Dijkstra } sempre dir amb K
Serà *perce* BFS

b) $A \in \text{NP}$ $\{ \langle x \rangle \mid \exists y \ A(x, y) \}$
 $x = \text{entrada}$
 $y = \text{testimoni (output)}$
 $A = \text{verificador polinòmic}$

$y \equiv$ camí correcte o i ocals en la

$A \equiv$ comprova x té un camí simple amb longitud $\geq k$

Hamiltonian path $\equiv$ LPATH amb $k = n$!

$\left(\begin{array}{l} \text{Si } k \text{ són aretes} \Rightarrow k = n-1 \\ \text{Si } k \text{ són vèrtexs} \Rightarrow k = n \end{array} \right)$

5. Consider the problem MODULAR EXPONENTIATION: given $a, b, n \in \mathbb{N}$, compute $d = a^b \bmod n$.
Can MODULAR EXPONENTIATION be solved in polynomial time?

$$a \cdot b \bmod n = (a \bmod n) \cdot (b \bmod n) \bmod n$$

↓
mod per
a no fer
que veni
tant

$$a \cdot a \cdot a \cdot a$$

$$a^5 = a \cdot a^2 \cdot a^2$$

Si que pot ser polinomi - pr 1 !!

6. **Half clique.** Considereu el problema SUBGRAF COMPLET D'ORDRE MEITAT (HALF-CLIQUE):
Donat un graf no dirigit G , decidir si conté una $\lfloor \frac{n}{2} \rfloor$ -clique.

(a) Demostreu que HALF-CLIQUE $\in$ NP

(b) Demostreu que si HALF-CLIQUE $\in$ P, aleshores CLIQUE $\in$ P.

a) $y =$ conjunt

verificar = comprovar tot junt

b) Reducció

$$x \in A \iff f(x) \in B$$

$\lfloor \frac{n}{2} \rfloor$ -Clique

↓
Clique

• $k = \lfloor \frac{n}{2} \rfloor$ ja tenim nodes

• $k < \lfloor \frac{n}{2} \rfloor$ cal afegir vertèbres
fins a n

$$\bullet k > \left\lfloor \frac{n}{2} \right\rfloor \quad \text{ignarem nodes que no estiguen correctes i ho fem si} \quad \left(\frac{n+5}{2} \right) = k$$

7. **Self reducibility SAT.** Suposem que tenim un algorisme que decideix SAT en temps polinòmic. Dissenyeu un algorisme tal que donada una fórmula booleana F retorni una assignació que la satisfaci, si existeix. Demostreu que si $\text{SAT} \in P$ aleshores trobar una assignació que la satisfà també és computable en temps polinòmic. Aquesta propietat s'anomena *self-reducibility*.

$A()$

$$F = x_1 \dots x_n$$

Algorisme divergent

if ($A(F)$) return false (mirem si e' input)

else

for ($i = 1$; $i \leq n$; $++i$)

if ($A(F)_{x_i} = 1$) assignem 1 a x_i

else assignem 0 a x_i

invariant $\rightarrow$ la fórmula sempre serà satisfactible

8. **Self reducibility CLIQUE.** Suposem que tenim un algorisme que decideix CLIQUE en temps polinòmic. Dissenyeu un algorisme tal que donat un graf G i donat un natural k , retorni una clíca de k vèrtexs, si existeix. Demostreu que si CLIQUE $\in P$ aleshores trobar una clique també és computable en temps polinòmic. Aquesta propietat s'anomena *self-reducibility*.

$A()$ polinòmic

G

k

Retornem clíca de k vèrtexs

Algorisme (diminuant)

if (not $A(G)$) return false

else

- iterem sobre els vèrtexs

- Fem $G' = G - \{v_i\}$

- Si $A(G')$ trobem v_i i fem $Z = G'$

return $V(G)$

(podem usar
ràtzons per què
sigui més eficient !!)

Funció !!

Exh.

- Si no està - l'eliminem i ja

- Si està, mirem veïns per veure si un d'ells no està / + eficient)

9. **Stingy SAT.** Let us consider the following problem:

STINGY SAT Given a set of clauses (each a disjunction of literals) and an integer k , find a satisfying assignment in which at most k variables are true, if such an assignment exists.

Prove that STINGY SAT is NP-complete.

$$F = (x_1 \vee x_2) \wedge (x_3 \vee x_4) \vee (x_5)$$

k

com a molt k variable are true

STINGY $\in$ NP

$$P \text{ també } \subseteq P$$

$$A \subseteq B$$

Si podem decidir NP $\in$ A

$$B \subseteq A$$

$\uparrow$
 $\in$ NP

$\uparrow$
podem veure
si $\in$ NP

• Testimoni: conjunt de variable a true, mida conjunt $\leq k$

• Verificador: comprovar si la assignació satisfi F

Reducció

$$S \subseteq V \Rightarrow \forall e = (u, v) \in E \quad [u \in S \vee v \in S]$$

Vertex Cover $\equiv$ conjunt de vèrtexs que cobreixen totes les arestes!

VertexCover $\leq_p$ STINGY-SAT

$$x \in \text{VertexCover} \iff f(x) \in \text{STINGY-SAT}$$

• Té G un rec. vèrtexs com $\leq k$ vèrtexs? (Vertex Cover)

- Definim la funció $f(k)$ com una transformació que agafa un vèrtex $x \in V$ i crea una variable o la fórmula F .

La fórmula seria de l'estil $F = \bigwedge_{\forall e=(u,v)} (x_u \vee x_v)$, k

$$\forall k \ (G, k) \Leftrightarrow S\text{-SAT}(F, k)$$

$$S \rightarrow x_u = \begin{cases} 1 & \text{si } u \in S \\ 0 & \text{si } u \notin S \end{cases}$$

$$X \rightarrow S = \{x_i \mid x_i = 1\}$$

2-SAT i monotona (només +) , i és NP-complet per a $k \geq 3$!!!

$$\text{ Clique } = \{ \langle G, k \rangle \mid \exists S \subseteq V \quad P(G, k, S) \}$$

$$\text{ Clique-3 } = \{ \langle G, k \rangle \mid \exists S \subseteq V \quad P(G, k, S) \wedge G \text{ té grau } \leq 3 \}$$

10. **At most 3.** Consider the CLIQUE problem restricted to graphs in which every vertex has degree at most 3. Call this problem CLIQUE-3. $\Rightarrow \in P$ *polinomial!*

(a) Prove that CLIQUE-3 is in NP.

(b) What is wrong with the following proof of NP-completeness for CLIQUE-3? We know that the CLIQUE problem in general graphs is NP-complete, so it is enough to present a reduction from CLIQUE-3 to CLIQUE. Given a graph G with vertices of degree ≤ 3 , and a parameter k , the reduction leaves the graph and parameter unchanged: clearly the output for the reduction is a possible input for the CLIQUE problem. Furthermore, the answer to both problems is identical. This proves the correctness of the reductions, and, therefore, the NP-completeness of CLIQUE-3.

graph ≤ 3
 $\Downarrow$
 clique on V vertices!

(c) It is true the the VERTEX COVER problem remains NP-complete even when restricted to graphs in which every vertex has degree at most 3. Call this problem VC-3. What is wrong in the following proof of NP-completeness for CLIQUE-3?

We present a reduction from VC-3 to CLIQUE-3. Given a graph $G = (V, E)$ with node degrees bounded by 3, and a parameter b , we create an instance of CLIQUE-3 by leaving the graph unchanged and switching the parameter to $|V| - b$. Now, a subset $C \subseteq V$ is a vertex cover in G if and only if the complementary set $V - C$ is a clique in G . Therefore G has a vertex cover of size $\leq b$ if and only if it has a clique of size $\geq |V| - b$. This proves the correctness of the reduction and, consequently, the NP-completeness of CLIQUE-3.

(d) Describe an $O(|V|)$ algorithm for CLIQUE-3.

a) • Testimoni: conjunt de vèrtex (jo t'he'n grau ≤ 3 segons entn)
 • Verificar: comprovar si el conjunt forma un clique

b) Què és el que està malament?

Doncs que està mal parat! Ho ve's en CLIQUE $\in P$ CLIQUE-3

c) VC-3 is NP-complete?

Si S és un VC, $V - S$ és un conj. independent